

A1 Report

Team members: Ellen Chan (101303431), Joanna Joy (101303246)

1.

(b.1)

```
1 def minimizeL1(X, y):
2     # first d variables are w, second n variables are delta
3     n, d = X.shape
4
5     c = np.concatenate([np.zeros(n), np.ones(d)])
6
7     G = np.concatenate([np.concatenate([np.zeros_like(X), -np.eye(n)], axis=1),
8                               np.concatenate([X, -np.eye(n)], axis=1),
9                               np.concatenate([X, -np.eye(n)], axis=1)
10                              ])
11
12     h = np.concatenate([np.zeros_like(y), y, -y])
13
14     cvxopt.solvers.options["show_progress"] = False
15     res = cvxopt.solvers.lp(c, G, h)
16     w = res['x'][-d:]
17     return np.array(w)
```

Bugs:

- Line 5: The input for np.zeros should be d and the input for np.ones should be n because according to slide 21 of 3105_L1_LinReg-post, $c^T = [0, 0, \dots, 0, 1, 1, \dots, 1]$ has d zeros and n ones
- Line 9: The X should be -X because $G^{(31)} = -X$ in LP formulation for L1 loss
- Line 15: The input matrices should be cvxopt.matrix instead of NumPy arrays, because cvxopt.solvers.lp only accepts cvxopt.matrix as inputs.
- Line 16: w should equal res['x'][:d] because res['x'][-d:] only gets the last d elements instead of the first d elements since $x = [w \ \delta]$ where w has dimensions $dx1$

(c.1)

$c = [0, 0, \dots, 0, 1]$ with d zeros

(c.2)

$G^{(1)} = [0_{1 \times d}, -1]$
 $h^{(1)} = 0$

(c.3)

$G^{(2)} = [X, -1_n]$
 $h^{(2)} = y$

(c.4)

$$G^{(3)} = [-X, -1_n]$$

$$h^{(3)} = -y$$

(d.1)

Table 1 - train loss

Model	L_2 Loss	L_1 Loss	L_∞ Loss
L_2 Model	0.10495042	0.24041132	1.57458049
L_1 Model	0.12636625	0.19189811	1.92762751
L_∞ Model	0.3037852	0.51410012	0.86867345

Table 2 - test loss

Model	L_2 Loss	L_1 Loss	L_∞ Loss
L_2 Model	0.05225069	0.24198996	1.04570912
L_1 Model	0.02819978	0.18878822	0.819
L_∞ Model	0.35015872	0.54065975	2.15221648

(d.2)

These are our initial observations: the L_1 loss column has the largest numbers, while L_∞ has the lowest. In addition to this, the L_∞ model usually seems to have the highest loss, while L_1 usually seems to have the lowest.

Between the columns, the values for L_1 loss are expected to be the highest since the calculation for L_1 loss is the sum of $|\hat{y}_i - y_i|$ for each data point i . L_2 loss is lower since it is calculated by the sum of $\frac{1}{2}(\hat{y} - y)^2$ for each data point i , which results in a lower average value than L_1 . L_∞ loss has the lowest values, because it is calculated by the $\max \{|\hat{y}_i - y_i|\}$ for all data points i , which doesn't take the sum of all differences for each data point.

The L_∞ model numbers are aligned with the concept that the larger the "p" value in L_p loss, the more influence outliers will have. Since the training data was autogenerated, there is a high chance for outliers, which explains the high averages. The same concept can be applied to L_1 loss, since p is the lowest among the losses we calculated. We also noted that L_2 falls in between the other models.

When comparing the two table data, we see that test loss has greater variance among the averages. Train loss data still exemplifies the concepts we discussed above, but to a far less noticeable degree, and in some cases, the opposite of what was expected. This changed in the test loss data, which during generation had significantly more sets of data output. The

more data that is provided, the stronger the association with the “p” rule. This highlights how L_∞ doesn’t seem to be a highly favored model.

(e.2)

Table 1 - train loss

Model	L_2 Loss	L_1 Loss	L_∞ Loss
L_2 Model	53.3834382	8.18704596	32.32866981
L_1 Model	58.0707062	7.9922139	41.44364034
L_∞ Model	66.95248316	9.39624932	26.49548145

Table 2 - test loss

Model	L_2 Loss	L_1 Loss	L_∞ Loss
L_2 Model	54.96060837	8.30806942	33.25938522
L_1 Model	60.44826502	8.23565737	41.36420576
L_∞ Model	68.7830664	9.47698309	36.97856775

2.

(b.1)

$$- \log(\sigma(Xw)) = np.logaddexp(0, -Xw)$$

$$- \log(\sigma(Xw)) = \log(e^0 + e^{-Xw})$$

$$- \log\left(\frac{1}{1+e^{-Xw}}\right) = \log(e^0 + e^{-Xw})$$

$$\log\left(\left(\frac{1}{1+e^{-Xw}}\right)^{-1}\right) = \log(e^0 + e^{-Xw})$$

$$\log(1 + e^{-Xw}) = \log(e^0 + e^{-Xw})$$

$$\log(1 + e^{-Xw}) = \log(1 + e^{-Xw})$$

(b.2)

$$- \log(1_n - \sigma(Xw)) = np.logaddexp(Xw, 0)$$

$$- \log(1_n - \sigma(Xw)) = \log(e^{Xw} + e^0)$$

$$- \log(1_n - \sigma(Xw)) = \log(e^{Xw} + 1)$$

$$- \log\left(1_n - \frac{1}{1+e^{-Xw}}\right) = \log(e^{Xw} + 1)$$

$$- \log\left(\frac{1+e^{-Xw}}{1+e^{-Xw}} - \frac{1}{1+e^{-Xw}}\right) = \log(e^{Xw} + 1)$$

$$\begin{aligned}
- \log\left(\frac{e^{-Xw}}{1+e^{-Xw}}\right) &= \log(e^{Xw} + 1) \\
\log\left(\left(\frac{e^{-Xw}}{1+e^{-Xw}}\right)^{-1}\right) &= \log(e^{Xw} + 1) \\
\log\left(\frac{1+e^{-Xw}}{e^{-Xw}}\right) &= \log(e^{Xw} + 1) \\
\log\left(\frac{1}{e^{-Xw}} + \frac{e^{-Xw}}{e^{-Xw}}\right) &= \log(e^{Xw} + 1) \\
\log\left(\frac{1}{e^{-Xw}} + 1\right) &= \log(e^{Xw} + 1) \\
\log\left(\frac{1}{(e^{Xw})^{-1}} + 1\right) &= \log(e^{Xw} + 1) \\
\log(e^{Xw} + 1) &= \log(e^{Xw} + 1)
\end{aligned}$$

(c.1)

Table 5: Training accuracies with different hyper-parameters

m	Train Accuracy	dim1	Train Accuracy	dim2	Train Accuracy
10	0.977	1	0.8467	1	0.9272
50	0.9356	2	0.9301	2	0.9254
100	0.9229	4	0.9868	4	0.9267
200	0.92355	8	1.0	8	0.937

Table 6: Test accuracies with different hyper-parameters

m	Test Accuracy	dim1	Test Accuracy	dim2	Test Accuracy
10	0.8691	1	0.83528	1	0.91982
50	0.91215	2	0.91672	2	0.9173
100	0.91697	4	0.96826	4	0.91318
200	0.91867	8	0.99456	8	0.90937

(c.2)

As the number of data points (m) increases, the test accuracy increases while train accuracy slightly decreases. The decrease is likely because the training set is small, which can make the fitted accuracy look unusually high. For instance, when m=10, there are only 20 training examples.

As the number of dim1 features increases, both training and test accuracies increase. This occurs since the values corresponding to dim 1 features will be shifted by +1 for class 0 and by -1 for class 1, thus increasing the amount of information to distinguish the classes.

As the number of dim2 features increases, the training accuracy stays mostly consistent, while the test accuracy decreases slightly. This is expected because dim2 features carry the randomly generated values without shift to indicate class. Therefore, they just add noise to the dim1 features, which slightly worsens generalization.

(d.2)

BCW Dataset

Average Training Accuracy: 1.0

Average Test Accuracy: 0.9473684210526316